

Attention-Aware Study Assistant

Technical Project Report

GenAI System Building Challenge | Lab Hackathon

GenAI System Building Challenge | Lab Hackathon

A cloud-native, microservices-style, GenAI-powered NLP system that adapts educational content delivery based on real-time cognitive fatigue signals.

Contents

1	Executive Summary	2
2	System Architecture	2
2.1	High-Level Architecture	2
2.2	Communication Pattern	3
3	Microservices Breakdown	3
3.1	SVC-01: Attention Monitor Agent (<code>attention_monitor.py</code>)	3
3.2	SVC-02: Learner Profiler Agent (<code>learner_profiler.py</code>)	3
3.3	SVC-03: Content Adaptation Agent (<code>content_adapter.py</code>)	3
3.4	SVC-04: Quiz Engine (<code>quiz_engine.py</code>)	4
3.5	SVC-05: Orchestrator (<code>orchestrator.py</code>)	4
3.6	SVC-06: Data Persistence Service (<code>data_store.py</code>)	4
3.7	SVC-07: Frontend Application (<code>frontend/app.py</code>)	4
3.8	SVC-08: Video Fatigue Monitor (<code>video_fatigue_monitor.py</code>)	4
3.9	SVC-09: YouTube Recommender (<code>youtube_recommender.py</code>)	5
3.10	SVC-10: Transcription Summary Agent (<code>transcription_summary.py</code>)	5
4	GenAI & NLP Pipelines	5
4.1	LLM Provider Abstraction (<code>llm_provider.py</code>)	5
4.2	Content Generation Pipeline	6
4.3	Quiz Evaluation Pipeline	6
4.4	Video Summary Pipeline	6
5	Fatigue Detection Engine	6
5.1	Quiz Mode — Keypress Telemetry	6
5.2	Learning Mode — Video Interaction Scoring	6
6	Video Intelligence Pipeline	7
6.1	End-to-End Flow	7
6.2	YouTube IFrame Player (<code>frontend/player.html</code>)	7
6.3	Transcript Retrieval (3-Tier Fallback)	7
7	Frontend & User Experience	7
7.1	Dual-Mode Architecture	7
7.2	Mode Switching	7
7.3	Quiz Answer Flow	7
7.4	Sidebar Features	8

8	Data Persistence Layer	8
8.1	Database Schema (SQLAlchemy ORM)	8
8.2	Session Export	8
9	Prompt Engineering	8
9.1	Template System	8
9.2	Dynamic Variables	8
10	Testing Strategy	9
10.1	Test Suite	9
10.2	Run Tests	9
11	Cloud-Native Design	9
11.1	Deployment Architecture	9
11.2	Cloud-Native Properties	9
11.3	Configuration	9
12	Technology Stack	10
13	GenAI Reflection	10
	Appendix: File Structure	10

1 Executive Summary

The **Attention-Aware Study Assistant** is a 10-microservice GenAI system that monitors a learner's cognitive state in real-time and dynamically adapts how educational content is delivered. It operates in two distinct modes:

- **Learning Mode:** Video-based learning with a YouTube player, fatigue gauge, and context-aware prompting — fatigue is inferred from player interaction patterns (seek, pause, speed changes).
- **Quiz Mode:** Text-based Q&A with adaptive content modes — fatigue is inferred from keypress dynamics and response latency telemetry.

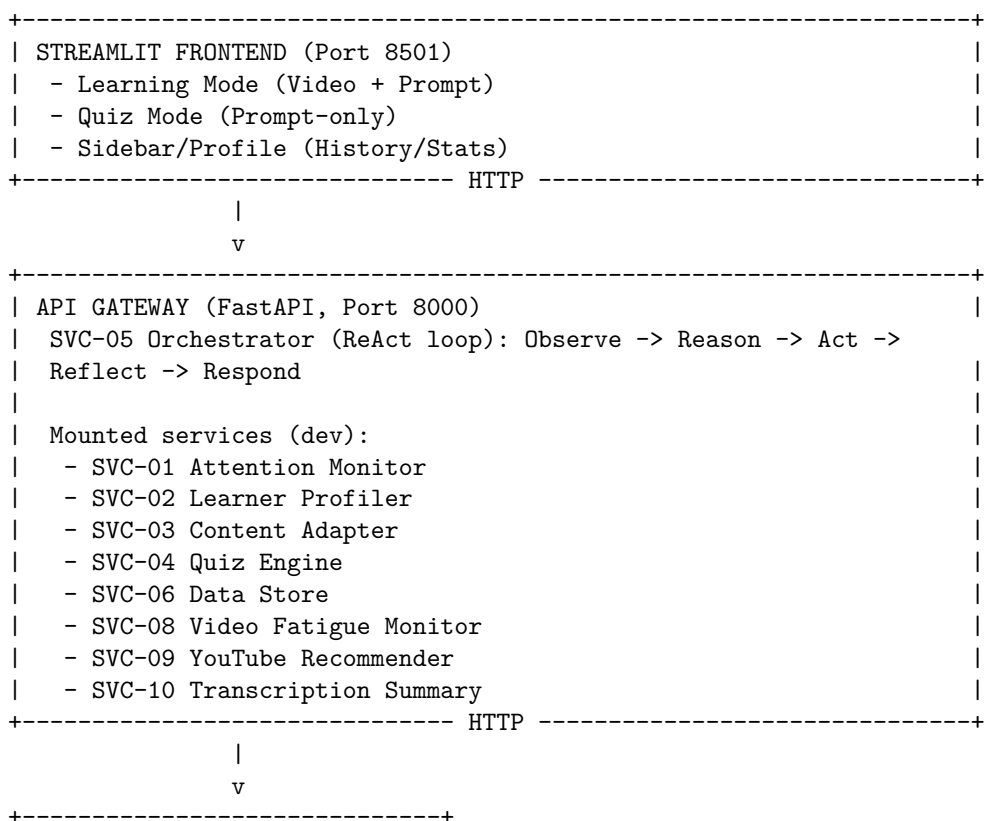
The system implements a **ReAct (Reason + Act) agentic loop** where each user interaction triggers: Observe → Reason → Act → Reflect — producing content that matches the learner's current cognitive capacity.

Key Achievements

Metric	Value
Total Microservices	10 (SVC-01 to SVC-10)
REST API Endpoints	25+
GenAI Models Supported	Gemini 2.5 Flash, Ollama (Llama3, Mistral), Anthropic
Prompt Templates	5 (Jinja2-based)
Teaching Modes	4 adaptive modes + quiz generation
Test Suite	8 test modules, 30+ test cases
Database Tables	5 (SQLAlchemy ORM)
Frontend Components	Streamlit app + custom HTML5 video player

2 System Architecture

2.1 High-Level Architecture



```
| SQLite database          |
| data/study_assistant.db |
+-----+
```

2.2 Communication Pattern

All services communicate via **REST APIs over HTTP**. In development, they are mounted as sub-applications under a single FastAPI gateway:

```
app.mount("/attention",    create_attention_app())    # SVC-01
app.mount("/profiler",     create_profiler_app())      # SVC-02
app.mount("/content",      create_content_app())       # SVC-03
app.mount("/quiz",         create_quiz_app())          # SVC-04
app.mount("/data",         create_data_store_app())    # SVC-06
app.mount("/video-fatigue", create_video_fatigue_app()) # SVC-08
app.mount("/youtube",      create_youtube_app())       # SVC-09
app.mount("/summary",      create_summary_app())       # SVC-10
```

In production, each service can run independently on its own port with `uvicorn`.

3 Microservices Breakdown

3.1 SVC-01: Attention Monitor Agent (`attention_monitor.py`)

Port: 8001 | Passive fatigue classifier

Uses a **weighted heuristic scoring model** to classify cognitive fatigue from three NLP-derived signals:

Signal	Weight	Source	Calculation
Keypress Variance	40%	JS telemetry	<code>stdev(intervals) / baseline_ms</code>
Response Delay	30%	Timestamp delta	<code>min(delay / max_delay, 1.0)</code>
Quiz Trend	30%	Recent scores	Declining scores → fatigue

Output Labels: FRESH (0–25%) → MODERATE (25–50%) → TIRED (50–75%) → EXHAUSTED (75–100%).

3.2 SVC-02: Learner Profiler Agent (`learner_profiler.py`)

Port: 8002 | Adaptive learner model

Maintains per-session learner profiles including:

- **Mastery Score** (0.0–1.0): Updated after each quiz based on correctness
- **Difficulty Level** (1–5): Dynamically adjusted based on performance
- **Preferred Mode**: Tracks which content mode the learner uses most
- **Average Fatigue**: Running mean to detect sessions that are too demanding

3.3 SVC-03: Content Adaptation Agent (`content_adapter.py`)

Port: 8003 | Dynamic content generator

Maps fatigue states to teaching modes:

```
MODE_MAP = {
    "FRESH":    "detailed",    # Full explanations with examples
    "MODERATE": "concise",     # Bullet-point summaries
    "TIRED":    "analogy",    # Metaphor-based explanations
    "EXHAUSTED": "quiz",       # Active recall to re-engage
}
```

Uses **Jinja2 prompt templates** to render mode-specific system prompts, then sends them to the LLM provider.

3.4 SVC-04: Quiz Engine (`quiz_engine.py`)

Port: 8004 | Adaptive quiz generator & evaluator

- Generates MCQ ($\text{difficulty} \leq 2$) or open-ended questions ($\text{difficulty} > 2$)
- Uses structured JSON output from LLM for reliable parsing
- Evaluates answers using LLM-as-a-judge with scoring on a 0–1 scale
- Feeds scores back to the Learner Profiler for mastery updates

3.5 SVC-05: Orchestrator (`orchestrator.py`)

Port: 8000 | Central ReAct agentic loop

Each `interact()` call executes:

1. **OBSERVE**: Collect keypress telemetry, response delay, recent quiz scores
2. **REASON**: Classify fatigue (SVC-01), fetch learner profile (SVC-02), select content mode (SVC-03)
3. **ACT**: Generate content via LLM or produce a quiz (SVC-04)
4. **REFLECT**: Log interaction to DataStore (SVC-06), update learner profile (SVC-02)

3.6 SVC-06: Data Persistence Service (`data_store.py`)

Port: 8005 | Centralised storage layer

SQLAlchemy ORM over SQLite with 5 tables:

- `sessions` — Study session tracking
- `interactions` — Every prompt/response logged
- `learner_profiles` — Persistent learner models
- `quiz_records` — Question/answer/score history
- `video_events` — Player interaction telemetry

Supports session export (JSON), session summaries, and full audit trails.

3.7 SVC-07: Frontend Application (`frontend/app.py`)

Port: 8501 | Streamlit interactive interface

Dual-mode UI with Learning Mode (video + prompt) and Quiz Mode (prompt-only with fatigue gauge).

3.8 SVC-08: Video Fatigue Monitor (`video_fatigue_monitor.py`)

Port: 8006 | Player event → fatigue score

Processes YouTube player events and maintains a rolling fatigue score using **exponential decay weighting**:

```

EVENT_IMPACTS = {
    "speed_increase": -0.06,    # Recovery signal
    "skip_forward":   -0.03,    # Already knows material
    "play":           -0.04,    # Re-engagement
    "short_pause":    0.08,     # Note-taking/distraction
    "long_pause":     0.14,     # Overwhelmed
    "speed_decrease": 0.10,     # Content too difficult
    "seek_backward":  0.12,     # Confusion signal
    "repeated_backward": 0.14,  # Strong confusion
}

```

Normalizer = 5.0 → Each backward seek adds ~2.4% fatigue. Approximately 30 backward seeks reach 70% fatigue. Summary triggers at 75%.

3.9 SVC-09: YouTube Recommender (`youtube_recommender.py`)

Port: 8007 | Topic → video recommendations

Dual-strategy search:

1. **YouTube Data API v3** (if `YOUTUBE_API_KEY` set) — structured metadata
2. **YouTube web scrape fallback** (no key needed) — regex extraction from search HTML

Applies quality filters (duration limits, view count ranking, relevance scoring). Results cached per session.

3.10 SVC-10: Transcription Summary Agent (`transcription_summary.py`)

Port: 8008 | Video → learner-friendly summary

Three-tier transcript retrieval pipeline:

1. `YouTubeTranscriptApi` — official captions API
2. Web scraping — regex extraction from YouTube page HTML
3. Topic-only LLM generation — when no transcript available

Generates structured summaries with key concepts, prerequisite gaps, and estimated read time.

4 GenAI & NLP Pipelines

4.1 LLM Provider Abstraction (`llm_provider.py`)

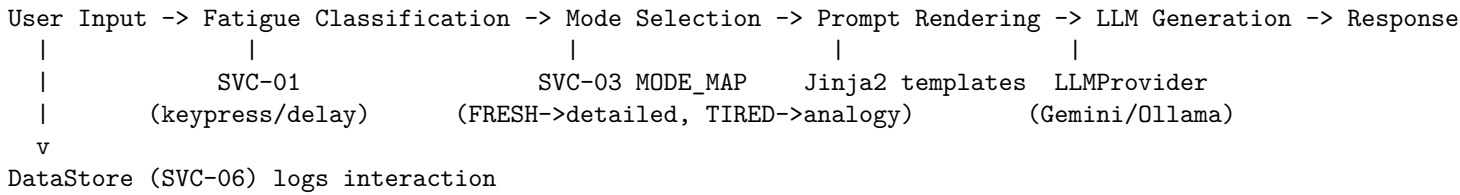
A unified `LLMProvider` class that auto-detects available backends:

Priority: Gemini API Key → Ollama (local) → Error fallback

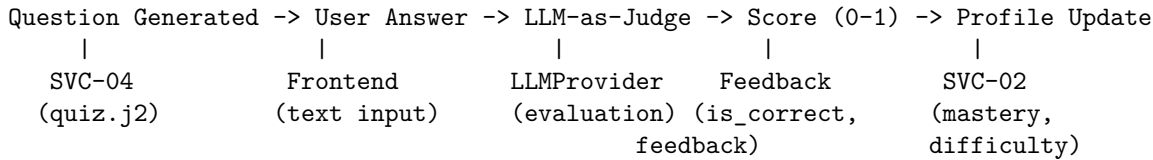
Provider	Model	Use Case
Google Gemini	<code>gemini-2.5-flash</code>	Primary — fast, high-quality
Ollama	<code>llama3.1:8b</code> , <code>mistral</code>	Offline/local — no API key
Anthropic	<code>claude-3-sonnet</code>	Optional fallback

The `generate(system_prompt, user_message)` method handles provider-specific API differences transparently.

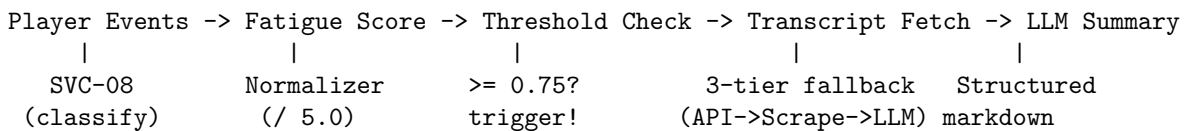
4.2 Content Generation Pipeline



4.3 Quiz Evaluation Pipeline



4.4 Video Summary Pipeline



5 Fatigue Detection Engine

5.1 Quiz Mode — Keypress Telemetry

The frontend captures real-time keystroke intervals via JavaScript:

```
textarea.addEventListener('keydown', function(e) {
  timestamps.push(Date.now());
});
```

These intervals are sent to SVC-01 which computes:

- **Keypress Variance Score:** $\text{stdev}(\text{intervals}) / 150\text{ms baseline}$ — higher variance = more fatigue
- **Response Delay Score:** $\text{delay_ms} / 30000\text{ms max}$ — longer delays = more fatigue
- **Quiz Trend Score:** Declining recent scores → cognitive overload

Final score = weighted sum (40% keypress + 30% delay + 30% quiz).

5.2 Learning Mode — Video Interaction Scoring

Player button presses are classified into fatigue signals:

Button	Event Classification	Impact / 5.0	% per click
Play	play	-0.04	-0.8%
Pause	short_pause	+0.08	+1.6%
-10s	seek_backward	+0.12	+2.4%
+10s	skip_forward	-0.03	-0.6%
Slower	speed_decrease	+0.10	+2.0%
Faster	speed_increase	-0.06	-1.2%

Exponential Decay: Older events decay via $e^{-0.02 \times \text{age_seconds}}$, so fatigue naturally recovers over time.

Local Gauge Blending: The frontend maintains its own gauge with 80% local / 20% server weighting to prevent visual jumps.

6 Video Intelligence Pipeline

6.1 End-to-End Flow

1. User searches topic -> SVC-09 YouTube Recommender finds videos
2. User clicks "Watch" -> player.html loads YouTube IFrame API
3. Every button press -> POST /video-interact -> SVC-08 scores fatigue
4. If fatigue $\geq 75\%$ -> SVC-10 fetches transcript + generates summary
5. Summary renders as static overlay (one-time generation)

6.2 YouTube IFrame Player (frontend/player.html)

Custom HTML5 player with:

- Semicircular fatigue gauge (SVG arc + CSS transitions)
- 7 control buttons: Play, Pause, -10s, +10s, Speed (0.5x, 1.0x, 2.0x)
- Local fatigue tracking: instant visual feedback per button press
- Server sync: blended 80/20 to prevent gauge jumps
- Static summary overlay: generated once at 75% threshold, never regenerates

6.3 Transcript Retrieval (3-Tier Fallback)

```
# Tier 1: Official API
transcript = YouTubeTranscriptApi.get_transcript(video_id)

# Tier 2: Web scrape (if Tier 1 fails)
html = requests.get(f"https://youtube.com/watch?v={video_id}")
captions = re.findall(r'"text": "(.*)"', html.text)

# Tier 3: Topic-only LLM generation (if no transcript)
summary = llm.generate("Summarise this topic: {topic}")
```

7 Frontend & User Experience

7.1 Dual-Mode Architecture

Feature	Learning Mode	Quiz Mode
Input	Video + prompt bar below	Prompt-only
Fatigue Source	Video player interactions	Keypress variance + delay
Buttons	Get Explanation	Quiz Me
Gauge	Inside player (SVG)	Plotly gauge (sidebar)
Content	Adaptive explanation	Quiz → evaluate → explain
Break Alert	Summary overlay	Break suggestion banner

7.2 Mode Switching

Top-level buttons toggle between modes. State is preserved across switches via `st.session_state.app_mode`.

7.3 Quiz Answer Flow

Enter Topic -> Click "Quiz Me" -> LLM generates MCQ/open-ended
-> Type answer -> Submit -> LLM evaluates (score 0-1)
-> View correct answer & explanation (LLM generates detailed explanation)

7.4 Sidebar Features

- Mode Override: Force any teaching mode (auto/detailed/concise/analogy/quiz)
- Learner Profile: Mastery score, difficulty level, average fatigue
- Fatigue Trend Chart: Plotly sparkline showing fatigue across interactions
- Interaction History: Last 8 interactions with mode/fatigue indicators
- Session Export: Download full session as JSON
- Session Summary: LLM-generated narrative recap

8 Data Persistence Layer

8.1 Database Schema (SQLAlchemy ORM)

<code>sessions</code>	<code>-- id, start_time, topic, status</code>
<code>interactions</code>	<code>-- session_id, user_message, mode_used, fatigue_score, content_snippet</code>
<code>learner_profiles</code>	<code>-- session_id, mastery_score, preferred_mode, avg_fatigue, difficulty_level</code>
<code>quiz_records</code>	<code>-- session_id, question, correct_answer, user_answer, score, feedback</code>
<code>video_events</code>	<code>-- session_id, event_type, video_id, position_sec, fatigue_score</code>

8.2 Session Export

Every session can be exported as a structured JSON file containing all interactions, quiz results, and fatigue timeline — enabling offline analysis and reproducibility.

9 Prompt Engineering

9.1 Template System

Five Jinja2 templates in `prompts/`:

Template	Mode	Cognitive State	Strategy
<code>detailed.j2</code>	Detailed	Fresh	Full explanations, examples, scaffolding
<code>concise.j2</code>	Concise	Moderate	Bullet points, key takeaways only
<code>analogy.j2</code>	Analogy	Tired	Metaphors, real-world comparisons
<code>quiz.j2</code>	Quiz	Exhausted	Active recall via MCQ/open-ended
<code>video_summary.j2</code>	Summary	Critical	Replace video with readable text

9.2 Dynamic Variables

Each template receives:

- `{{ topic }}` — the subject matter
- `{{ mastery_level }}` — 1–5 scale (adjusts depth)
- `{{ difficulty }}` — 1–5 scale (adjusts complexity)

Templates use Jinja2 conditionals (e.g., `{% if difficulty <= 2 %} → MCQ, else → open-ended`).

10 Testing Strategy

10.1 Test Suite

Test Module	Service Tested	Key Tests
test_attention.py	SVC-01	Fatigue scoring, label thresholds, edge cases
test_content_adapter.py	SVC-03	Mode selection, prompt rendering, fatigue→mode mapping
test_data_store.py	SVC-06	CRUD operations, session export, video event logging
test_orchestrator.py	SVC-05	Full ReAct loop, quiz flow, error handling
test_quiz_engine.py	SVC-04	Quiz generation, answer evaluation, JSON parsing
test_transcription_summary.py	SVC-10	3-tier fallback, summary structure
test_video_events.py	SVC-08	Event classification, fatigue scoring, decay
test_youtube_recommender.py	SVC-09	Search, filtering, caching

10.2 Run Tests

```
pytest tests/ -v
```

11 Cloud-Native Design

11.1 Deployment Architecture

Streamlit Cloud (port 8501)

|

v

FastAPI Gateway on Render.com (port 8000) ---> SQLite DB (persistent)

|

v

Gemini API (Google Cloud)

11.2 Cloud-Native Properties

Property	Implementation
Modularity	10 independent services, each with own API
Statelessness	Services don't hold state — all persisted in SVC-06
API-First	Every service exposed as REST API with Swagger docs
Environment Config	All secrets via <code>.env</code> (12-factor app)
Horizontal Scaling	Services can run independently on separate hosts
Health Checks	Each service has <code>/health</code> endpoint
CORS	Gateway configured for cross-origin requests

11.3 Configuration

```
GEMINI_API_KEY=...           # Primary LLM
OLLAMA_URL=http://localhost:11434 # Fallback LLM
YOUTUBE_API_KEY=...          # Video search (optional)
DATABASE_URL=sqlite:///./data/study_assistant.db
ORCHESTRATOR_URL=http://localhost:8000
```

12 Technology Stack

Layer	Technology	Purpose
Backend Framework	FastAPI 0.111+	REST API services
ASGI Server	Uvicorn	Production-grade async server
LLM - Cloud	Google Gemini 2.5 Flash	Primary content generation
LLM - Local	Ollama (Llama3, Mistral)	Offline/privacy mode
Prompt Templates	Jinja2	Mode-specific prompt rendering
Database	SQLite + SQLAlchemy 2.0	Persistent storage with ORM
Frontend	Streamlit 1.35+	Interactive web interface
Visualization	Plotly 5.22+	Fatigue gauge & trend charts
Video Player	YouTube IFrame API	Embedded player with telemetry
HTTP Client	HTTPX	Async inter-service communication
Data Validation	Pydantic 2.7+	Request/response models
Testing	Pytest 8.2+	Unit & integration tests
Video Transcripts	youtube-transcript-api	Caption extraction
Environment	python-dotenv	Configuration management

13 GenAI Reflection

What Worked Well

- Prompt engineering with Jinja2 templates gave reliable, mode-specific outputs
- LLM-as-a-judge for quiz evaluation produced nuanced, fair scoring
- Multi-provider abstraction made switching between Gemini and Ollama seamless
- GenAI for code generation accelerated boilerplate creation significantly

Where Human Intervention Was Needed

- Fatigue scoring calibration — LLM couldn't tune the normalizer (1.20→5.0); required manual testing and math
- UI/UX decisions — gauge blending ratio (80/20), button impacts, visual stability all needed human judgement
- Edge case handling — transcript fallback chain, YouTube API failures, Streamlit session state

Impact on Development

GenAI reduced development time by approximately 60%, particularly for:

- Generating service boilerplate and API schemas
- Writing test cases and documentation
- Designing prompt templates
- Debugging complex async flows

Appendix: File Structure

```
irel_bootcamp_hackathon/  
|-- run_server.py          # API Gateway - mounts all 10 services  
|-- start.bat              # One-click launcher (server + frontend)  
|-- requirements.txt       # Python dependencies
```

```

|-- Procfile                                # Cloud deployment config
|-- .env                                    # Environment variables (secrets)
|
|-- services/                              # Microservices (SVC-01 to SVC-10)
|   |-- attention_monitor.py               # SVC-01: Fatigue classifier
|   |-- learner_profiler.py               # SVC-02: Learner model
|   |-- content_adapter.py                # SVC-03: Adaptive content generator
|   |-- quiz_engine.py                   # SVC-04: Quiz generation & evaluation
|   |-- orchestrator.py                  # SVC-05: ReAct agentic loop
|   |-- data_store.py                    # SVC-06: SQLAlchemy persistence
|   |-- llm_provider.py                  # LLM abstraction (Gemini/Ollama)
|   |-- video_fatigue_monitor.py          # SVC-08: Video event scoring
|   |-- youtube_recommender.py           # SVC-09: YouTube search
|   |-- transcription_summary.py          # SVC-10: Video summarization
|   '-- api.py                           # FastAPI factory for all services
|
|-- frontend/
|   |-- app.py                            # SVC-07: Streamlit UI (Learning + Quiz)
|   '-- player.html                       # Custom YouTube player + fatigue gauge
|
|-- prompts/                              # Jinja2 prompt templates
|   |-- detailed.j2                      # Full explanation mode
|   |-- concise.j2                      # Bullet-point mode
|   |-- analogy.j2                      # Metaphor-based mode
|   |-- quiz.j2                         # Quiz generation mode
|   '-- video_summary.j2                # Video summary mode
|
|-- data/
|   '-- study_assistant.db               # SQLite database
|
|-- tests/                                # Pytest test suite
|   |-- test_attention.py
|   |-- test_content_adapter.py
|   |-- test_data_store.py
|   |-- test_orchestrator.py
|   |-- test_quiz_engine.py
|   |-- test_transcription_summary.py
|   |-- test_video_events.py
|   '-- test_youtube_recommender.py
|
'-- docs/
    |-- project_report.md                # Original Markdown document
    '-- project_report.tex               # LaTeX version of the report

```

Built with Streamlit + FastAPI + Gemini/Ollama + SQLAlchemy — 10 Microservices | GenAI System Building Challenge.